

ARAB ACADEMY FOR SCIENCE, TECHNOLOGY & MARITIME TRANSPORT

College of Engineering and Technology
Computer Engineering Department

PROJECT DOCUMENTATION

Embedded Systems

Car Anti-Theft Security System

STM32 Blackpill + ESP32 + GSM/GPS Integration

Supervised By	Dr. Amr Fahmy
Course	Embedded Systems
Semester	8
Academic Year	2025/2026

1. Project Team

1.1 Team Members

Full Name	College ID	Role / Responsibility	Email
Alhussain Aly Ayad	221004147	Team Leader	Alhussain_Ayad@hotmail.com
Mohamed Tamer Taymour	221004807	member	mohtam7504@gmail.com
Seifeldin Tarek Ismail	221006919	member	seiftarek211200@gmail.com
Shapinam Abdelfattah Hassan	221004292	member	shapinam.abdelfattah@gmail.com
Hanya Essam Abdelfattah	221006161	member	Hanyaessam2050@gmail.com

1.2 Sub-Group Distribution

Sub-Group	Module / Layer	Key Deliverable
Hardware	STM32 wiring, sensors, H-Bridge	Fully wired prototype
Firmware	STM32 FreeRTOS code, RFID, state machine	Working STM32 firmware
GSM / GPS	GSM SMS alerts, GPS location tracking	SMS with live coordinates
Web Dashboard	ESP32 WiFi server, HTML dashboard	Live web UI on local network
Testing & Docs	System testing, documentation	Test report & final documentation

2. Introduction & Problem Statement

2.1 Background

Modern vehicle security relies heavily on embedded systems to monitor sensors and control hardware. This project presents a complete, multi-layered anti-theft system built around an STM32 microcontroller. Because the system must handle multiple hardware components at the same time, it uses FreeRTOS to manage tasks efficiently and without delay.

The physical security layer includes an RFID scanner for keyless authorization, a magnetic reed switch to monitor the driver's door, and a vibration sensor to detect glass breaking. Upon successful RFID authorization, the start engine button is safe to be pushed to make an H-bridge drive a motor to simulate the engine starting. To connect the vehicle to the user, the system features a dual-communication setup: an ESP32 microcontroller hosts a web interface for remote control over Wi-Fi, while a GSM module sends backup SMS alerts if the internet is unavailable. Finally, a GPS module tracks the car's location, and an LCD screen displays real-time system states for debugging.

2.2 Problem Statement

Standard car alarms are easily ignored. This local approach is outdated because it fails to alert the owner if they are far away from the vehicle. Additionally, while newer "smart" cars offer mobile apps, they often rely entirely on an internet connection, leaving the car vulnerable in network dead zones.

Specifically, current basic systems fail to address:

- **Distance Limitations:** Local alarms do not actively notify the owner during a breach.
- **Lack of Redundancy:** Relying on a single communication method (like just Wi-Fi) creates a single point of failure.
- **Missing Telemetry:** Owners cannot see the exact location of the car or know exactly which sensor triggered the alarm.

There is a need for an embedded system that physically secures the vehicle while actively sending real-time data, coordinates, and alerts to the user through both online and offline channels.

2.3 Motivation

The primary motivation for this project is to build a highly reliable IoT security system that gives the user complete remote control and awareness of their vehicle.

Technically, this project serves as a practical implementation of hardware integration and task scheduling. By using FreeRTOS on an STM32, the system ensures that high-priority tasks, like triggering the buzzer and LED alarms when the door switch is forced, happen instantly, without being blocked by slower tasks like updating the LCD or sending a GPS coordinate. The ultimate goal is to create a robust prototype that combines local sensor security with global connectivity, ensuring the owner is always informed and in control.

3. Project Objectives

The primary goal of this project is to design, develop, and implement a comprehensive, IoT-enabled automotive security system utilizing embedded microcontrollers and a Real-Time Operating System (FreeRTOS). To achieve this overarching goal, the project is broken down into the following specific technical and functional objectives:

3.1 Hardware Integration and local Security

- **To implement multi-layered physical access control:** Utilize an RFID scanner as the primary physical authorization method to differentiate between the vehicle owner and unauthorized individuals.
- **To establish a localized perimeter defense:** Integrate a magnetic reed switch and a vibration sensor to actively monitor the vehicle's doors and windows for forced entry or tampering.
- **To simulate vehicle immobilization and ignition:** Control a DC motor via an H-bridge motor driver to physically demonstrate the engine starting only after successful user authentication.
- **To provide immediate audio-visual deterrents:** Trigger localized hardware alarms, including high-intensity LEDs and a buzzer, instantly upon detecting unauthorized access.

3.2 System Architecture and RTOS Implementation

- **To utilize FreeRTOS for deterministic multitasking:** Manage multiple concurrent hardware operations on the STM32 microcontroller, ensuring high-priority security interrupts (like sensor triggers) are processed immediately without being delayed by secondary tasks.
- **To establish reliable serial communication:** Design a robust UART communication protocol between the primary STM32 microcontroller and the secondary ESP32 module to continuously exchange system states (Armed/Disarmed, Door, Engine, Alarm) and GPS coordinates.

3.3 Remote Monitoring and Telemetry (IoT Integration)

- **To develop a globally accessible web interface:** Program the ESP32 to act as a web server that hosts an interactive HTML/JS dashboard, allowing the user to monitor the vehicle's state from any web-enabled device.
- **To implement real-time GPS tracking:** Parse coordinate data from a hardware GPS module and integrate it with an embedded OpenStreetMap interface on the web dashboard to provide live, visually smoothed vehicle tracking.
- **To establish reliable cloud connectivity and global telemetry:** Configure the ESP32 to maintain a persistent Wi-Fi connection (Station Mode) that bridges the local hardware to a global MQTT broker (HiveMQ Cloud). Implement automatic network reconnection algorithms to recover gracefully from Wi-Fi drops, alongside MQTT Last Will and Testament (LWT) protocols to accurately broadcast the vehicle's online/offline status to the remote web dashboard.
- **To provide redundant offline alerts:** Integrate a GSM module to dispatch immediate SMS notifications to the owner's mobile phone during an intrusion, ensuring the alarm system functions even in network dead zones.

3.4 User Interaction & Control

- **To enable two-way remote management:** Allow the user to send "Force Arm" and "Disarm" commands directly from the web interface to the STM32 via REST API endpoints.
- **To provide transparent system diagnostics:** Display real-time debugging information on an integrated hardware LCD and monitor the ESP32's free heap memory directly on the web dashboard to ensure system stability and performance.

4. Technology Stack

This chapter outlines the complete hardware and software ecosystem required to build the smart vehicle security prototype. The architecture is divided into the physical components that form the embedded perimeter and the software/cloud infrastructure that handles telemetry, messaging, and remote user interaction.

4.1 Hardware Components

Component	Model / Spec	Purpose
Main Microcontroller	STM32F411 BlackPill	Primary control unit managing the RTOS, hardware interrupts, and real-time sensor polling.
WiFi / Web Module (microcontroller)	ESP32 38-pin	Dedicated IoT coprocessor managing Wi-Fi, TLS encryption, and MQTT cloud communication.
GSM Module	SIM800L / V2-Link	Provides redundant cellular connectivity for offline SMS alerts.
GPS Module	NEO-6M / compatible	Captures real-time geographical coordinates for live vehicle tracking.
RFID Reader	MFRC522 (SPI)	Primary authentication hardware for secure, localized keyless entry.
LCD Display	16x2 I2C (hd44780)	Real-time system state display
Motor + H-Bridge	DC Motor + L298N	Simulates engine ignition; powered via 12V and controlled safely by a relay or H-bridge.
Magnetic Reed Switch	2-pin NC/NO	Mounted on the driver's door to detect physical unauthorized entry.
Vibration Sensor	SW-420	Acts as a glass-break detector monitoring the vehicle's structural perimeter.
Push Button	2-leg momentary	Engine start/stop trigger after authentication
Buzzer	Active buzzer	Provides immediate, localized audio deterrents during a security breach.
LED	5mm standard	Provides immediate, localized visual deterrents during a security breach.
DC-DC Buck Converter	LM2596S	Steps down the 12V battery supply to a stable, isolated 5V to safely power the STM32 and ESP32 logic boards.
12V Battery		Main power supply delivering high current to drive the simulated engine motor.

4.2 Software & Libraries

Layer	Technology / Library	Purpose
Operating System	FreeRTOS (STM32)	Manages concurrent hardware tasks deterministically, prioritizing time-sensitive security interrupts.
Development IDE	Arduino IDE 2.x	The primary integrated development environment used for writing, compiling, and flashing firmware.
Board Package	STM32duino Core	The Arduino-compatible framework bridging the standard C++ API to the STM32 ARM Cortex-M hardware.
Hardware Drivers	MFRC522 Library	Handles the SPI communication protocol required to interface with the RC522 RFID reader module.
Hardware Drivers	hd44780	A highly efficient, FreeRTOS-safe driver for managing I2C communications with the local LCD.
Hardware Drivers	TinyGPS++	Parses raw NMEA sentences from the GPS module's UART stream into usable latitude and longitude doubles.
Local Communication	UART Protocol	The serial communication standard used to bridge the STM32 and ESP32 to share parsed string commands.
Network Connectivity	WiFi.h & WiFiClientSecure.h	Manages ESP32 Wi-Fi connections and establishes an encrypted TLS layer for secure cloud communication.
IoT Messaging	PubSubClient.h	Connects the ESP32 to the HiveMQ Cloud broker to publish telemetry and subscribe to remote commands.
Data Serialization	ArduinoJson.h	Serializes embedded system states into formatted JSON payloads for efficient cloud transmission.
Frontend UI	HTML / CSS / Vanilla JS	The deployed remote dashboard, featuring a responsive grid, login modal, and interactive map.
Frontend Mapping	OpenStreetMap Embed	Renders the live vehicle coordinates pushed from the GPS module within the web dashboard.

Layer	Technology / Library	Purpose
Frontend Networking	MQTT.js	A JavaScript library enabling the web application to connect to the HiveMQ broker via Secure WebSockets (WSS).
Cloud Protocol	MQTT (HiveMQ Cloud)	A lightweight publish-subscribe messaging protocol handling two-way global communication.

5. System Architecture & Design

5.1 High-Level Architecture

The system is built around a distributed dual-microcontroller architecture communicating over UART serial. The STM32 acts as the primary real-time security controller, it manages all physical sensors, the RFID reader, the motor, buzzer, LED, GPS module, and GSM module. The ESP32 acts as a dedicated IoT gateway and MQTT bridge. Rather than hosting a local web page, the ESP32 connects securely to a HiveMQ Cloud broker over TLS, allowing a separately deployed web dashboard to monitor and control the vehicle globally via Secure WebSockets (WSS).

The communication flow between the system components is as follows:

Source	→	Destination	Protocol	Data
Sensors / RFID / Button	→	STM32	GPIO / SPI / I2C	Door state, vibration, card UID, button press
GPS Module	→	STM32	UART 9600	NMEA sentences → latitude/longitude
STM32	→	GSM Module	UART 9600	AT commands, SMS with GPS location
STM32	→	ESP32	UART 9600	STATUS, GPS, DOOR, ENGINE, NET every 500ms
ESP32	→	HiveMQ Broker	MQTT / TLS	Publishes JSON telemetry and online status
HiveMQ Broker	→	Web Dashboard	WSS (WebSockets)	Real-time state updates to the user interface
Web Dashboard	→	HiveMQ Broker	WSS (WebSockets)	CMD:ARM / CMD:DISARM commands

Source	→	Destination	Protocol	Data
HiveMQ Broker	→	ESP32	MQTT / TLS	Forwards commands to the ESP32 via subscription
ESP32	→	STM32	UART 9600	Forwards CMD:ARM / CMD:DISARM to the controller

5.2 STM32 Firmware Architecture

The STM32 firmware is structured around a finite state machine managed by two concurrent FreeRTOS tasks to ensure deterministic execution:

- **mainTask (Priority 1):** Handles all real-time sensor polling (reed switch, vibration sensor, push button), RFID card reading, state transitions, and serial communication parsing with the ESP32. This task runs on a fast 10ms cycle.
- **smsSendTask (Priority 2):** Handles all GSM operations, including network health monitoring every 30 seconds with automatic recovery, and non-blocking SMS dispatching when an alarm triggers. The higher priority ensures SMS operations are not starved, while FreeRTOS delays ensure the sensor polling continues to run.

State Machine Transitions:

Current State	Trigger	New State	Action
ARMED	Authorized RFID card	DISARMED	Motor off, buzzer off, LED off
ARMED	Door open / vibration / button press	ALARM	Buzzer on, LED on, SMS triggered
ARMED	Unauthorized RFID card	ALARM	Buzzer on, LED on, SMS triggered
DISARMED	Push button pressed	ENGINE_ON	Motor starts via H-Bridge
DISARMED	Authorized RFID card	ARMED	Re-arm system
ENGINE_ON	Push button pressed	DISARMED	Motor stops, RFID reset
ENGINE_ON	Authorized RFID card	ARMED	Motor stops, system armed
ALARM	Authorized RFID card	DISARMED	Buzzer off, LED off, SMS cancel

5.3 ESP32 Cloud & Dashboard Architecture

The ESP32 has been upgraded from a local web server to a secure MQTT bridge. It connects to a HiveMQ Cloud cluster and facilitates real-time, two-way communication using a publish/subscribe model. The deployed web application utilizes MQTT.js to connect to the same broker via Secure WebSockets, eliminating the need for HTTP polling and providing instantaneous UI updates.

The ESP32 communicates over the following MQTT topics:

Topic	Direction	Description
carsec/<DEVICE_ID>/state	Publish (Retained)	Broadcasts full system state JSON (status, door, engine, net, lat, lon, heap) at 1 Hz.
carsec/<DEVICE_ID>/alarm	Publish (Retained)	Publishes a "1" immediately on an intrusion edge to trigger UI notifications.
carsec/<DEVICE_ID>/online	Publish (Retained)	Publishes "true" on boot. Configured with a Last Will and Testament (LWT) to broadcast "false" if the device disconnects unexpectedly.
carsec/<DEVICE_ID>/cmd	Subscribe	Listens for string payloads ("ARM" or "DISARM") dispatched from the web dashboard.

The web dashboard is a globally accessible single-page application. GPS location is displayed via an embedded OpenStreetMap iframe that smoothly updates upon receiving new coordinates. Alarm states trigger CSS blinking animations and slide-in notifications instantly upon receiving the MQTT message.

5.4 Pin Assignment

Signal	STM32 Pin	Direction	Connected To
RFID SS (CS)	PA4	Output	MFRC522 SDA
RFID RST	PA3	Output	MFRC522 RST
SPI (SCK/MOSI/MISO)	PA5/PA7/PA6	SPI Bus	MFRC522 SCK/MOSI/MISO

Signal	STM32 Pin	Direction	Connected To
I2C SDA/SCL	PB7/PB6	I2C Bus	LCD I2C adapter (0x27)
H-Bridge IN1/IN2	PB8/PB9	Output	L298N motor driver inputs
Buzzer	PA2	Output	Active buzzer
LED	PA1	Output	LED + resistor
Reed Switch (Door)	PB0	Input (PULLUP)	Magnetic door sensor
Vibration Sensor	PB1	Input (PULLDOWN)	SW-420 vibration module
Push Button	PB13	Input (PULLUP)	Momentary button, GND other leg
GSM TX/RX	PA11/PA12	UART	GSM module RX/TX
ESP32 TX/RX	PA9/PA10	UART	ESP32 RX/TX (cross-wired)
GPS TX	PB4	Input (UART RX)	GPS module TX

6. Project Steps & Phases

Phase	Step / Task	Expected Output / Deliverable
Phase 1	Requirements gathering, component selection, system block diagram	Component list, block diagram, approved scope
Phase 2	Hardware assembly, STM32 wiring, sensor connections, H-Bridge, LCD	Fully assembled hardware prototype
Phase 3	Basic STM32 firmware, GPIO, RFID, Button, LCD, state machine without RTOS	Working local state transitions (Arm/Disarm/Engine) on hardware
Phase 4	GPS integration, TinyGPS++ parsing, fix detection, coordinate reading	GPS fix shown on LCD, valid coordinates readable in serial
Phase 5	GSM integration, AT command testing, SMS send diagnostic task on LCD	Confirmed offline SMS delivery to the user's phone
Phase 6	FreeRTOS integration, concurrent SMS task + main task, hd44780 migration	Non-blocking SMS dispatching during normal real-time sensor polling
Phase 7	ESP32 MQTT Gateway, UART protocol bridging, Wi-Fi/TLS setup, HiveMQ pub/sub	Real-time hardware telemetry successfully published to the cloud broker
Phase 8	Web Dashboard Deployment, HTML UI, MQTT.js WebSockets, OpenStreetMap embed	Globally accessible, live-updating web dashboard connected to the broker
Phase 9	Network Resilience, GSM auto-reconnect, Wi-Fi failover, MQTT Last Will (LWT)	System successfully recovers from network drops without restarting or showing false states
Phase 10	System testing, bug fixes, final documentation	Final working prototype + complete engineering documentation

7. Group Task Distribution

Module / Feature	Assigned Member(s)	Sub-Group	Status
Hardware wiring & prototype assembly	All	Hardware	Completed
STM32 state machine & sensor logic	All	Firmware	Completed
RFID authentication	All	Firmware	Completed
FreeRTOS task design	All	Firmware	Completed
GPS module integration	All	GSM/GPS	Completed
GSM AT command interface & SMS	All	GSM/GPS	Completed
Network recovery & monitoring	All	GSM/GPS	Completed
ESP32 WiFi web server	All	Dashboard	Completed
HTML dashboard & OpenStreetMap	All	Dashboard	Completed
STM32 ↔ ESP32 UART protocol	All	Dashboard	Completed
System integration testing	All	Testing	Completed
Project documentation	All	Testing	Completed

8. Development Timeline

Week	Phase / Task	Deliverable
1	Requirements, component selection, block diagram	Component list + block diagram
2	Hardware assembly, sensor wiring, H-Bridge test	Assembled prototype
3	STM32 firmware, state machine, RFID, LCD	Working state transitions
4	GPS integration, TinyGPS++ parsing	GPS fix on LCD
5	GSM AT command testing, SMS diagnostic	Confirmed SMS to phone
6	FreeRTOS integration, hd44780 migration	Non-blocking concurrent tasks
7	ESP32 dashboard, WiFi server, HTML, API	Live web dashboard
8	Network recovery, button logic, full integration	Complete working system
9	System testing, bug fixes	Test report
10	Documentation, demo preparation, submission	Final documentation + demo

9. Full Project Documentation

9.1 Features & Functional Requirements

Hardware Security & Access

- **RFID Authorization:** System arms/disarms exclusively via pre-programmed RFID card UIDs.
- **Multi-Sensor Intrusion Detection:** Instant alarm triggers via magnetic door reed switches or physical vibration/impact.
- **Anti-Relay Engine Control:** Requires a physical button press to start the motor *after* RFID authentication, preventing remote cloning attacks. Unauthorized button presses trigger the alarm.
- **Audio-Visual Deterrents:** Immediate activation of an active buzzer and LED during an alarm state.

Connectivity & Telemetry

- **Global MQTT Web Dashboard:** An internet-hosted UI connects via HiveMQ Cloud, providing remote system status, global arm/disarm controls, and a live OpenStreetMap GPS tracker.
- **Redundant SMS Alerts:** The GSM module automatically dispatches an offline SMS containing intrusion details and Google Maps GPS coordinates directly to the owner.
- **Network Resilience:** GSM network connection is verified every 30 seconds with automatic re-registration. The ESP32 utilizes MQTT Last Will and Testament (LWT) to accurately report offline status.
- **Hardware Debugging:** A 16x2 I2C LCD outputs real-time states, network health, and satellite counts.

9.2 Non-Functional Requirements

- **Latency:** FreeRTOS polls sensors every 10ms, ensuring intrusion-to-alarm latency is under 50ms.
- **Concurrency:** High-priority FreeRTOS tasking guarantees that heavy operations (like 60-second SMS dispatching) never block real-time sensor polling or RFID reads.
- **Reliability:** The SMS protocol automatically retries every 15 seconds until network delivery (+CMGS) is confirmed.
- **Power Isolation:** Digital logic runs safely at 3.3V/5V via a Buck Converter, while the high-current DC motor is isolated and driven by a 12V H-Bridge.
- **Scalability:** The web dashboard uses Secure WebSockets (WSS) and MQTT.js, allowing real-time, zero-refresh updates across any modern mobile or desktop browser globally.

9.3 GSM AT Command Reference

STM32 utilizes the following core AT sequence to establish communication and dispatch alerts:

AT Command	Purpose	Expected Response
AT	Module alive check	OK
ATE0	Disable echo to prevent serial parsing errors	OK
AT+CREG?	Check network registration status	+CREG: 0,1 or 0,5
AT+COPS=0	Force automatic network operator selection	OK
AT+CSQ	Signal strength query (0-31, 99=no signal)	+CSQ: 18,0
AT+CMGF=1	Set SMS to text mode (not PDU)	OK
AT+CMGS="PHONE"	Initiate SMS to phone number	> (prompt)
(message) + Ctrl+Z	Send message content and terminate	+CMGS: XX

9.4 Web Dashboard UI/UX Notes

- **Design System:** CSS Grid layout (collapsing on mobile), utilizing a purple-blue gradient (667eea → #764ba2) with strict status colors (Green = Safe/Closed, Red = Alarm/Open).
- **Authentication:** Features a secure, browser-based modal for users to input their HiveMQ cluster credentials.
- **Real-Time Data (Push, Not Pull):** Replaced legacy HTTP interval polling with an MQTT WebSocket subscription. Changes to the vehicle's state update the DOM instantly.
- **Visual Alerts:** An active alarm state triggers CSS blinking animations and slide-in intrusion notifications instantly via the carsec/DEVICE_ID/alarm topic.

9.5 Testing Plan

Test Type	Method	Scope	Pass Criteria
Authentication	Physical	Scan valid/invalid RFID in all states.	Correct state transitions; invalid scans trigger alarm.
Sensor Alarms	Physical	Open reed switch or tap SW-420 while armed.	Alarm activates in < 50ms.
Button Logic	Physical	Press button in Armed vs. Disarmed states.	Armed = Alarm. Disarmed = Engine starts.
SMS Delivery	Software	Trigger alarm with SIM installed.	SMS received on target phone with valid coordinates.
Network Recovery	Physical	Remove/Reinsert SIM; disconnect Wi-Fi router.	GSM recovers in < 90s; ESP32 reconnects to broker.
MQTT Dashboard	Browser	Send Arm/Disarm commands via Web UI.	Vehicle responds instantly; Dashboard UI updates via Pub/Sub.
RTOS Concurrency	System	Scan RFID card <i>while</i> SMS is sending.	Card is read successfully; system does not freeze.

9.6 Known Limitations & Future Improvements

- **Wi-Fi Dependency for MQTT:** The current ESP32 architecture relies on a local Wi-Fi connection (or mobile hotspot) to reach the HiveMQ cloud. A future iteration will upgrade to an LTE/4G module (e.g., SIM7000) to handle both MQTT and SMS directly over cellular.
- **Single User Limitation:** The system currently hardcodes a single authorized RFID card UID. Future firmware will utilize EEPROM to support an "Admin Enrollment Mode" for multiple cards.
- **GPS Cold Starts:** The NEO-6M requires a clear-sky view and time to achieve a cold fix. Integrating Assisted-GPS (A-GPS) via the cellular module would drastically reduce lock times.
- **Engine Simulation:** The prototype uses an H-Bridge to simulate engine ignition. A production-ready model would replace the H-Bridge with automotive-grade relays spliced directly into the vehicle's starter and immobilizer circuits.
- **GSM Power Draw:** The SIM800L requires brief, high-current bursts (up to 2A) during transmission. A 1000µF bulk capacitor placed directly across the module's VCC/GND pins is highly recommended to prevent brown-out resets.

10. Submission Checklist

	Item
☒	Hardware prototype fully assembled and wired
☒	STM32 firmware compiles and uploads successfully
☒	FreeRTOS tasks running concurrently (main + SMS)
☒	All state machine transitions verified on hardware
☒	RFID authentication working for arm/disarm
☒	Door sensor and vibration sensor triggering alarm
☒	Push button engine start/stop logic working
☒	GPS module receiving fix and parsing coordinates
☒	GSM module sending SMS with GPS coordinates on alarm
☒	GSM network auto-recovery tested and working
☒	ESP32 successfully connects to Wi-Fi and HiveMQ Cloud broker via TLS
☒	MQTT Last Will and Testament (LWT) accurately updates device online/offline status
☒	Deployed web dashboard successfully connects to broker via Secure WebSockets
☒	Dashboard receives and displays live MQTT telemetry (status, GPS map, door, engine)
☒	Remote arm/disarm commands successfully published via MQTT and executed by STM32
☒	LCD shows correct state, network status, and GPS info
☒	Buzzer and LED activate/deactivate correctly
☒	Full system integration test passed
☒	Project documentation completed and formatted
☒	Live demo rehearsed by all team members

Github Link: <https://github.com/Alhussein-Ayad/carsec-dashboard.git>

Interface Link: <https://alhussein-ayad.github.io/carsec-dashboard/>